

Control de altura por realimentación negativa

Motor Dron TP — Sistemas Embebidos de Posgrado

Desarrollo de drone para control de cultivos y desmalezado

Plataforma STM32F446RE (Nucleo-F446RE)	IDE / HAL STM32CubeIDE · HAL STMicroelectronics
Interfaz remota Node-RED · Bluetooth HM-10	Tipo de control Lazo cerrado , PID

Punto de partida: El proyecto original

El firmware inicial consistía en un **único main.c** que combinaba dos periféricos independientes, sin ningún lazo de control entre ellos.

Experimento 1 — Control de velocidad motor DC

- ADC1 en **PA4 (canal 4)** con filtro de promedio móvil de 16 muestras
- PWM de **1 kHz** generado por TIM2_CH1 / PA5 → driver L298N (ENA)
- Sentido fijo adelante: IN1/IN2 en PB5/PB10
- Velocidad controlada directamente por el potenciómetro analógico

Experimento 2 — Medición de distancia HC-SR04

- TIM1 en modo **Input Capture**: TRIG en PA9, ECHO en PA8/TIM1_CH1
- Cálculo de ancho de pulso → distancia en cm
- Reporte por **USART2 a 115200 baud** al ST-Link (terminal serie)
- Frecuencia de muestreo: ~1 Hz

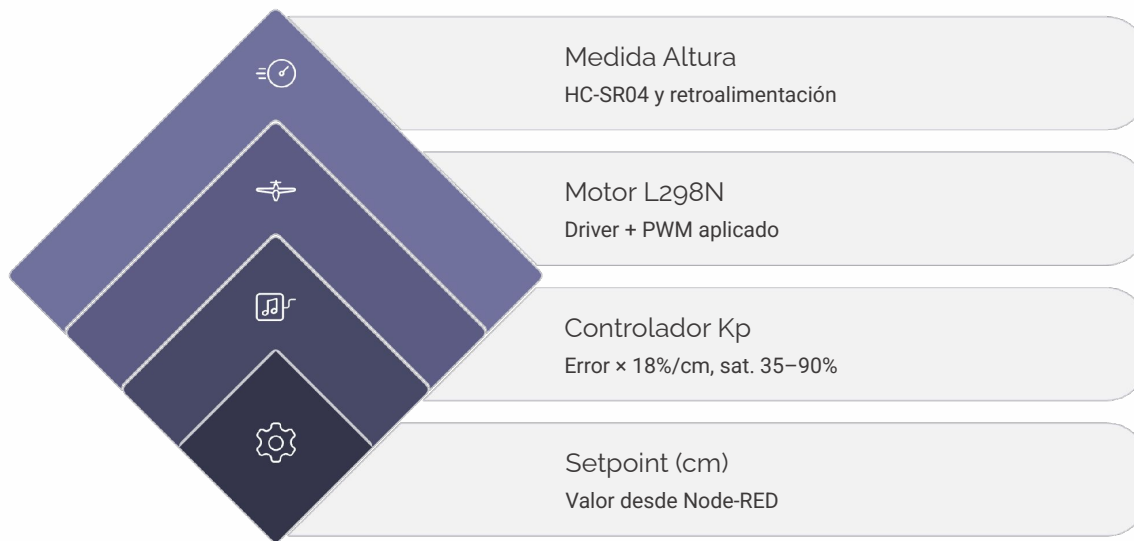
❌ **Punto crítico:** Motor y sensor NO estaban conectados entre sí. La distancia se medía pero no influía en la velocidad. No existía lazo de control, Bluetooth ni Node-RED.

Objetivo del rediseño

Transformar el banco de pruebas en un **verdadero sistema de control automático en lazo cerrado**, eliminando la entrada manual del potenciómetro y habilitando operación remota.

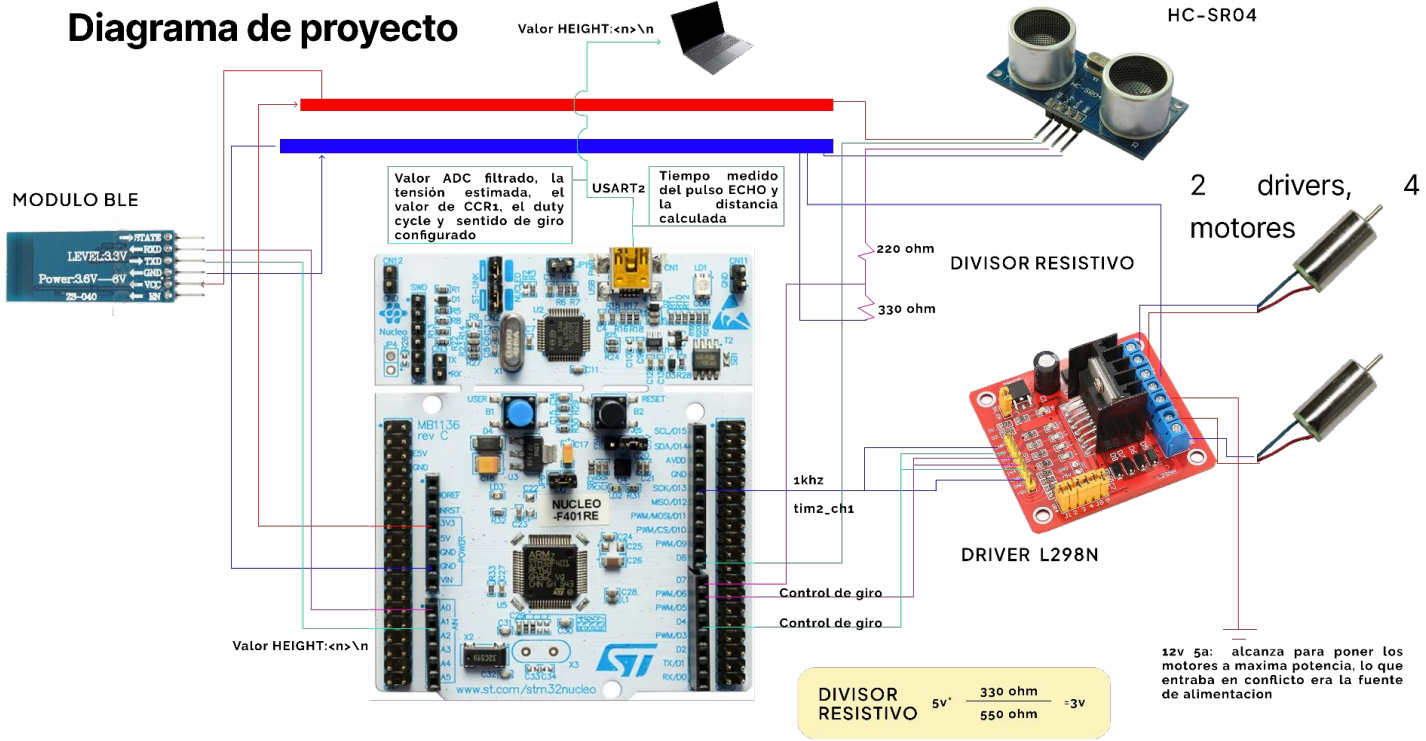
- 1 Retroalimentación real**
La distancia medida por el HC-SR04 se convierte en la variable de retroalimentación que ajusta automáticamente velocidad y sentido del motor en cada ciclo de control.
- 2 Setpoint remoto**
La altura objetivo (setpoint en cm) es configurable de forma remota por Bluetooth desde un dashboard Node-RED, sin recompilar ni reconectar el hardware.
- 3 Telemetría continua**
El sistema emite telemetría a 2 Hz (altura, setpoint, velocidad aplicada) hacia Node-RED para visualización en tiempo real y análisis de respuesta al escalón.

Diagrama de bloques — Lazo de realimentación negativa



El error se calcula como $e = \text{setpoint} - \text{altura_actual}$. Una zona muerta de ± 0.5 cm evita chattering por ruido de medición. El lazo opera a **10 Hz**; la telemetría se emite a 2 Hz de forma desacoplada.

Diagrama de proyecto



Arquitectura de software: La modularidad

La reorganización completa en módulos independientes (carpetas *Src/* e *Inc/*) es la decisión arquitectónica central del rediseño. Cada módulo tiene **responsabilidad única**, es testeable de forma aislada y reemplazable sin afectar al resto.



motor.c / motor.h

Driver L298N. Expone `Motor_Aplicar(sentido, velocidad_pct)` con enum `MOTOR_DETENER / MOTOR_SUBIR / MOTOR_BAJAR`. Oculta PWM (TIM2_CH1/PA5) y pines de dirección (PB5/PB10).



distance.c / distance.h

Driver HC-SR04 con Input Capture (TIM1). Filtro de promedio móvil circular de 8 muestras. Expone: `Distancia_Actualizar()`, `Distancia_ObtenerCm()`, `Distancia_UltimaFueValida()`.



height_control.c / height_control.h

Controlador proporcional puro.. Expone: `salida = EMPUJE_BASE + Kp·error + Ki·ferror + Kd·(-velocidad)`



hc10.c / hc10.h

Driver Bluetooth (UART4). Recepción por interrupción, parseo de `HEIGHT:<valor>`, envío de telemetría `H: , SP: , V:` a 2 Hz.



main.c — Orquestador

Solo inicializa periféricos y ejecuta el superloop. No contiene lógica de control ni de hardware. Estilo de registros crudos HAL/CMSIS.

Tabla de conexionado físico (pines)

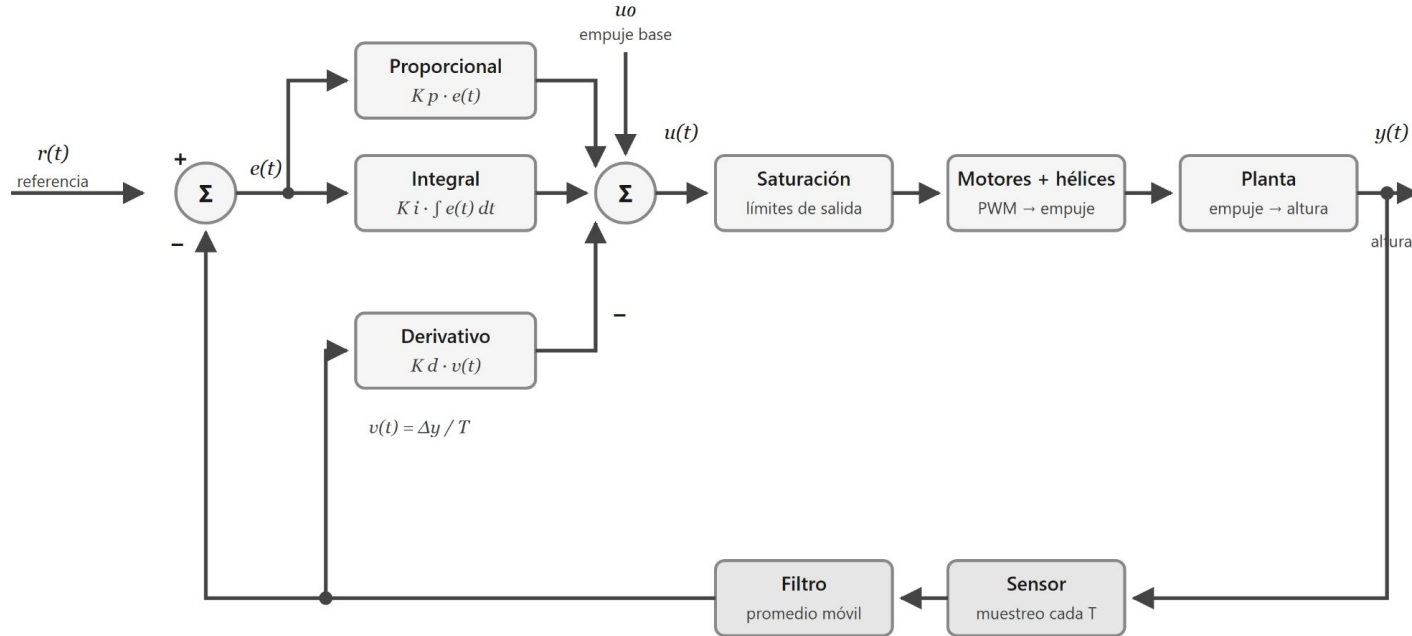
Asignación completa de pines del Nucleo-F446RE para el sistema integrado. **GND común obligatorio** entre fuente de motor y STM32 (falla real detectada y corregida en puesta en marcha).

Función	Pin Nucleo	Periférico / Notas	Subsistema
ENA (PWM)	PA5	TIM2_CH1, 1 kHz, duty cycle 35–90%	Motor L298N
IN1 (dirección)	PB5	GPIO salida digital	Motor L298N
IN2 (dirección)	PB10	GPIO salida digital	Motor L298N
TRIG	PA9	GPIO salida, pulso 10 µs	HC-SR04
ECHO	PA8	TIM1_CH1, Input Capture	HC-SR04
TX módulo BT	PA1	UART4_RX, 9600 baud	Bluetooth Hm-10 (bt05)
RX módulo BT	PA0	UART4_TX, 9600 baud	Bluetooth Hm-10 (bt05)
TX debug	PA2	USART2, 115200 baud (ST-Link)	Debug
RX debug	PA3	USART2, 115200 baud (ST-Link)	Debug

 USART2 y UART4 son periféricos separados deliberadamente para evitar contención entre tráfico de debug (115200 baud) y tráfico Bluetooth (9600 baud). Nunca comparten bus.

Sistema de control: Lazo de realimentación negativa

Ley de control proporcional implementada en `height_control1.c`. El controlador opera a **10 Hz** (cada 100 ms), frecuencia aumentada desde 1 Hz original para reducir overshoot.



$$e(t) = r(t) - y(t) \quad u(t) = \text{sat} [u_0 + K_p \cdot e(t) + K_i \cdot \int e(t) dt - K_d \cdot v(t)]$$

La rama derivativa parte de la salida medida, no del error: así un cambio de referencia no produce un pico en la derivada.

Sistema de control: Lazo de realimentación negativa

Ley de control proporcional implementada en `height_control1.c`. El controlador opera a **10 Hz** (cada 100 ms), frecuencia aumentada desde 1 Hz original para reducir overshoot.

Parámetros del controlador

0.4

K_p (%/cm)

Ganancia proporcional

0.2

K_i (%/cm)

Ganancia proporcional

0.9

K_d (%/cm)

Ganancia proporcional

10

Hz lazo

Frecuencia de control, velocidad máxima
de HC-SR04

± 0.5

Zona muerta (cm)

Evita vibraciones

Saturación mínima

88% – vence fricción e inercia

Saturación máxima

95% – margen de seguridad

Rango válido

3 a 20 cm (HC-SR04 confiable)

Comunicación Bluetooth: Protocolo y hardware

Enlace Bluetooth completo agregado al sistema. Implementado sobre **UART4 (PA0/PA1) a 9600 baud**. El hardware real identificado es un **HM-10** (breakout ZS-040, Bluetooth clásico/SPP).

Protocolo de entrada — Comando remoto desde Node-RED

```
HEIGHT:<valor_cm>\n
```

Ejemplo: HEIGHT:15\n fija el setpoint a 15 cm. El módulo hc10.c recibe por interrupción y parsea el token HEIGHT: antes de llamar a ControlAltura_FijarObjetivoCm().

Protocolo de salida — Telemetría a 2 Hz

```
H:<altura>,SP:<setpoint>,V:<velocidad_pct>\n
```

Puesta en marcha del módulo HM-10

Identificación del hardware

El módulo real es HM-10 (BT05). Diferencia crítica: utiliza la estructura de un modulo hc05 entonces la computadora no lo reconoce como ble y no puedo enviar comandos

Firmware puente standalone

Herramienta de soporte: firmware que usa el ST-Link como adaptador USB-TTL para enviar comandos AT al HM-10



USART2 emite simultáneamente mensajes legibles por humanos a 115200 baud para depuración en terminal serie, en paralelo al tráfico Bluetooth.

Dashboard en Node-RED: Interfaz de usuario remota

Flujo control-altura-dashboard.json — HMI/SCADA, permite ensayos sin recompilar y separa la visualización de la lógica de control embebida.

Grupo: Consigna

- Slider de **3 a 20 cm** para ajuste continuo del setpoint
- Botones de acceso rápido: escalón **5 cm** → **15 cm** para ensayos de identificación de lazo

Grupo: Respuesta al escalón

- Gráfico de línea: **altura medida vs. setpoint** en el tiempo
- Visualización de overshoot, tiempo de establecimiento y error en estado estacionario

Grupo: Velocidad vertical

- Gauge de **empuje aplicado (%)** en tiempo real
- Gráfico de velocidad vertical del motor

Nodo function — "Formatear HEIGHT"

```
// Arma string HEIGHT:<n>\n
// Valida rango 3-20 cm
// Replica hacia dos salidas:
//   → Puerto serie USB/ST-Link
//   → Puerto Bluetooth HC-05
// Sin cambiar firmware embebido
```

Nodo function — "Parsear telemetría"

```
// Interpreta línea recibida:
// H:12.5,SP:15,V:45\n
// Extrae: altura, setpoint, velocidad
// Alimenta gráficos y gauges
// Parser preparado para protocolo
// extendido: VZ, S, L (futuro Kd)
```

Valor K : Valores 01

Consigna

Altura objetivo (cm)

3

Escalon a 5 cm

Escalon a 15 cm

Empuje

85

%

0

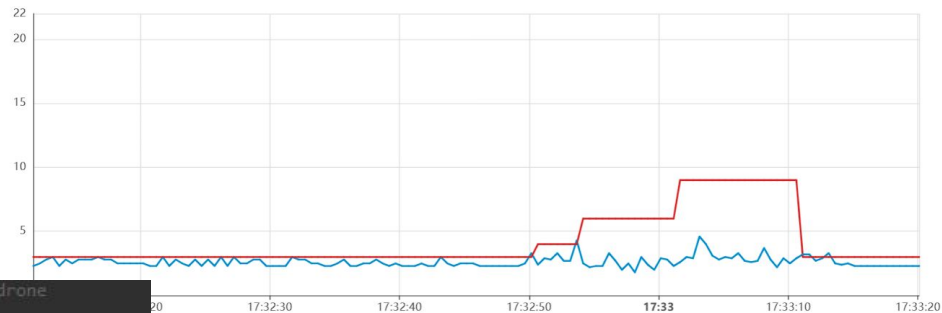
100

```
9 /* Duty al que el conjunto queda al borde de despegar, MEDIDO en el drone
0 * con pruebas, si cambia el peso o algo debería volver a medirse */
1 #define EMPUJE_BASE_PCT 88.0f
2
3 /* Ganancia proporcional: % de empuje por cada cm de error.
4 * hice algunas pruebas y por ahora esto es lo mejor, podría hacer una simulación */
5 #define KP_PCT_POR_CM 1f
6
7 /* Ganancia integral: % de empuje por cada cm*s de error acumulado. */
8 #define KI_PCT_POR_CM_S 0.05f
9
10 /* Ganancia derivativa: % de empuje por cada cm/s de velocidad vertical. */
11 #define KD_PCT_POR_CM_S 1.2f
12
13 /* Zona muerta sobre el término proporcional, lo fui probando
14 * hasta que dejó de temblar el empuje, le doy 0,5cm porque es
15 * el posible error */
16 #define ZONA_MUERTA_CM 0.5f
```

Respuesta al escalon

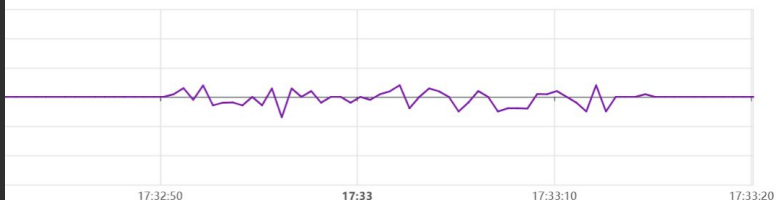
Altura vs Setpoint (cm)

Altura Setpoint

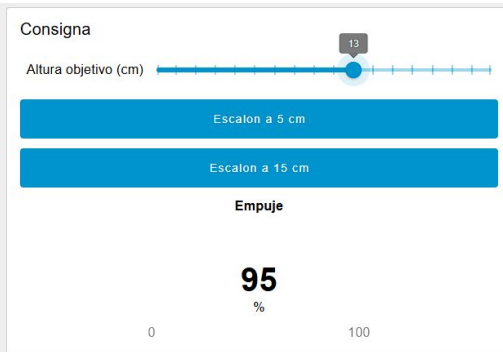


Velocidad vertical (cm/s)

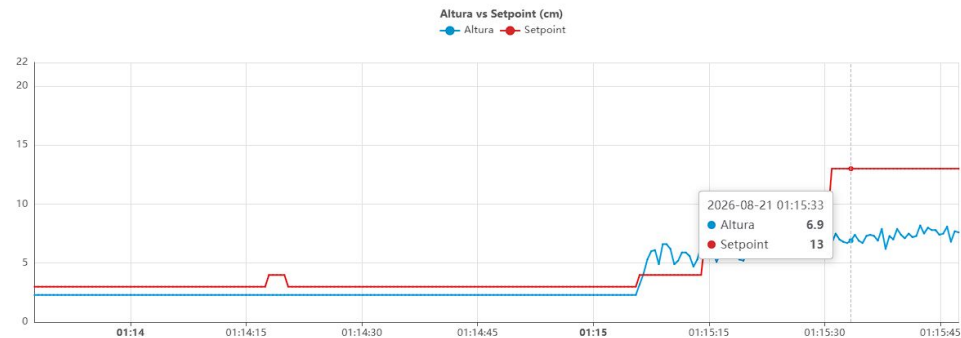
Velocidad



Valor K : Valores 02



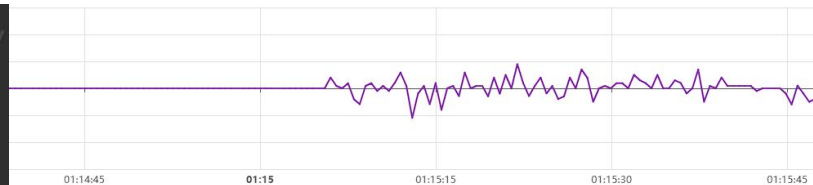
Respuesta al escalon



Velocidad vertical (para ajustar Kd)

Velocidad vertical (cm/s)

Velocidad



```
/* Ganancia proporcional: % de empuje por cada cm de error.
 * hice algunas pruebas y por ahora esto es lo mejor, podria hacer una simulacion */
#define KP_PCT_POR_CM 0.40f

/* Ganancia integral: % de empuje por cada cm*s de error acumulado. */
#define KI_PCT_POR_CM_S 0.2f

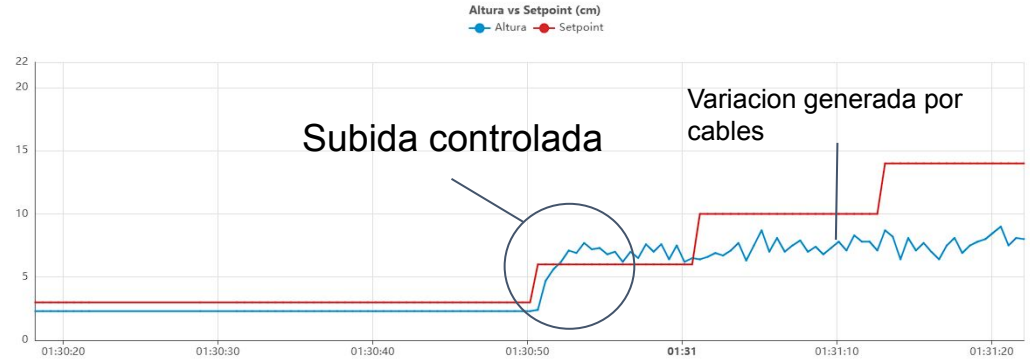
/* Ganancia derivativa: % de empuje por cada cm/s de velocidad vertical. */
#define KD_PCT_POR_CM_S 0.9f

/* Zona muerta sobre el termino proporcional, la fui probando
 * hasta que dejo de temblar el empuje, le doy 0,5cm porque es
 * el posible error */
#define ZONA_MUERTA_CM 0f Sin histeresis
```

Valor K : Valores 03



Respuesta al escalon



Velocidad vertical (para ajustar Kd)

```
/* Ganancia proporcional: % de empuje por cada cm de error.
 * hice algunas pruebas y por ahora esto es lo mejor, podria hacer una simulacion */
#define KP_PCT_POR_CM 0.40f

/* Ganancia integral: % de empuje por cada cm*s de error acumulado. */
#define KI_PCT_POR_CM_S 0.2f

/* Ganancia derivativa: % de empuje por cada cm/s de velocidad vertical. */
#define KD_PCT_POR_CM_S 0.9f

/* Zona muerta sobre el término proporcional. lo fui probando
 * hasta que deje de temblar el empuje, le doy 0,5cm porque es
 * el posible error */
#define ZONA_MUERTA_CM 0.5f
```

Con histeresis

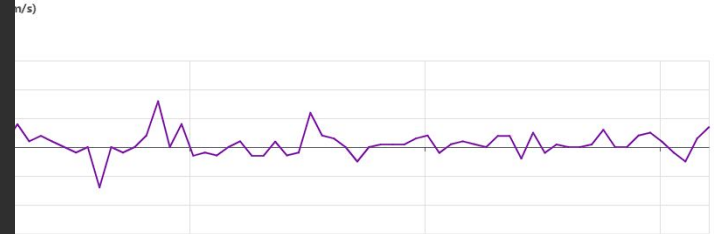


Tabla comparativa: Proyecto original vs. Rediseñado

El salto cualitativo entre ambas versiones abarca organización del código, arquitectura de control, comunicaciones e interfaz de usuario. La tabla resume las siete dimensiones de comparación.

Aspecto	Proyecto original	Proyecto rediseñado
Organización del código	Único <code>main.c</code> monolítico, sin separación de responsabilidades	Módulos separados: <code>motor.c</code> , <code>distance.c</code> , <code>height_control.c</code> , <code>hc10.c</code> , <code>main.c</code> orquestador
Entrada del sistema	Potenciómetro analógico (ADC manual, PA4)	Setpoint remoto por Bluetooth o directo desde dashboard Node-RED (HEIGHT: <n>\n)
Relación motor-sensor	Independientes, sin lazo. Distancia medida pero ignorada por el motor	Lazo cerrado de realimentación negativa con control proporcional ($K_p = 18\%$ /cm)
Frecuencia de actualización	Sensor a ~1 Hz, sin control automático	Control a 10 Hz , telemetría a 2 Hz , desacoplados
Comunicación	Solo USART2 a 115200 baud (debug local, terminal serie)	USART2 (debug) + UART4/HC-05 Bluetooth (comando y telemetría remota, 9600 baud)
Interfaz de usuario	Ninguna (terminal serie cruda, lectura manual)	Dashboard Node-RED: slider, botones de escalón, gráficos de respuesta, gauge de empuje
Alcance de motores	1 motor / 1 canal L298N en firmware único	1 motor en firmware principal + variante experimental de 4 motores (<code>variante_4_motores</code>) aislada del código de producción

Modularidad

De 1 archivo a 5 módulos con responsabilidad única, testeables y reemplazables de forma independiente.

Control automático

De velocidad manual por potenciómetro a lazo cerrado proporcional con zona muerta y saturación.

Telemetría remota

De terminal serie local a protocolo Bluetooth bidireccional con dashboard gráfico en tiempo real.

Extensibilidad

Roadmap visible: término derivativo K_d , protocolo extendido VZ/S/L, variante de 4 motores aislada.

Puntos de mejora

Control de motores

Descartar los drivers l298n y usar control ESC, que usa mosfet para hacer puentes H pero con mayor control de flujo de corriente

Control automático

Seguir configurando control de velocidad PID

Telemetría remota

Lograr controlar el drone por bluetooth

PID

Agregarle sistema de control con giroscopio, una vez ya configurado el pid lineal, empezar a controlar tambien giro